

从零训 Code LLM · Part 5: 转向 Gemma 4 Developer Agent——把评测环境搭起来

中文版 · V1 · 2026-10-05

0. 为什么有 Part 5

Part 1-4 从零预训了一个 0.6B 的 Transformer，做了 SFT，推了分布式训练和 RL 的理论。结论是：0.6B 的能力上限是模型规模决定的，不是技术问题。想让模型真正“解题”，应该换成**现成的强 base + post-train (LoRA / RL)**。

正好 Kaggle 开了 Google – The Gemma 4 Developer Agent Competition，base 固定、任务是 SWE-bench 式修 bug，天然就是 Part 4 那套 RL 理论的实操场。Part 5 记录从“读懂比赛”到“本地评测环境跑起第一个 task”的全过程：概念、架构和一路踩的坑。

两个赛道都报了：主赛道（final 12-02）和 Paper track（11-12）。定位是学 RL + agentic 为主，产出 paper / blog，不纠结名次。

1. 竞赛本质

- **Base 固定**：gemma-4-31b-it-qat-w4a16-ct，31B，4bit 权重 / 16bit 激活（QAT）。base 不在提交里，不能换。
- **任务**：agent 在离线沙箱里导航 Python repo → 定位 bug → 改代码 → 提交 git diff。判分是 SWE-bench 式的 PASS/FAIL，指标是 Resolution Rate。
- **公开题**：129 个 task，只来自 4 个 repo（fastapi / rich / requests / httpx），但每个 task 是该 repo 的不同 commit。Kaggle 评分用的是另一批隐藏题（私有 repo）。
- **提交物**：submission.zip < 3 GiB = agent.yaml（入口，纯声明式）+ prompts + sub_agents + skills + 可选 LoRA。不允许任何 Python 入口。
- **评测环境**：4×L4（共 96G），context 32K，默认每题 60 分钟 / 100 次 tool call / 500 turns。
- **两阶段评测**：Container A 里 agent 改代码、调 submit_patch；Container B 里打上 patch，把测试文件重置回 **baseline**（防作弊），再打官方 test_patch，跑 pytest。
- **9 个工具**：
run_command / read_file / edit_file / write_file / get_status / submit_patch /
get_code_neighbors / search_similar_code / get_code_subgraph。
- **坑点**：search_similar_code 是离线的，query 要传**符号名**（类名 / 函数名），不是自然语言——它拿 query 去 .npz 里匹配节点 key，再算余弦相似度。

2. RL 推演：为什么是 GRPO，多步怎么办

2.1 V、Q、A：critic 只估 V

- $V(s)$: 从状态 s 出发、按当前 policy 走到结束的期望总 reward。
- $Q(s, a)$: 在 s 先做动作 a ，之后按 policy 走到结束的期望总 reward。
- $A(s, a) = Q(s, a) - V(s)$: 这个动作比“平均水平”好多少。

actor-critic 里，policy 网络输出动作概率并采样，critic 网络估 $V(s)$ 。**Q 不需要单独的网络**，用“实际 reward + 下一状态的 V ”拼出来：

$$Q(s, a) \approx r + \gamma \cdot V(s'), \text{ 于是 } A \approx r + \gamma \cdot V(s') - V(s) \text{ (TD error)}$$

多步版本把 TD error 按 $\gamma\lambda$ 加权累加，就是 GAE（CarRacing 里手写过的那个）。

2.2 单步：V 退化成 baseline

单步任务（比如一次性生成代码）没有 s' ， $Q(s, a) = r$ 。此时 $V(s)$ 不是没有意义，而是退化成“所有动作 reward 的期望”，也就是 **baseline**：

$$A = r - \text{平均 reward}$$

GRPO 的做法：同一个 prompt 采 N 个回答，**用组内平均 reward 当 baseline**，不训 critic：

$$A_i = (r_i - \text{mean}(r)) / \text{std}(r)$$

配合 RLVR（可验证奖励）：代码题的 reward 直接由测试给出，连 reward model 也不用训。

2.3 本题只能 GRPO

提交里只有 `agent.yaml` + LoRA，也就是只有 policy；评测也只加载 policy 生成 patch。**没有地方放、也没法训一个 critic**，所以 PPO 不适用，只能用 GRPO 类的 critic-free 方法。

2.4 多步：轨迹级 reward，A 平摊到每一步

SWE agent 是多步 MDP：导航 → 读 → 改 → 测 → 修，reward 只在最后（patch 过不过）。GRPO 在多步下的做法：

1. 对同一个 task 采 N 条完整轨迹；
2. 每条轨迹一个 reward，组内算 baseline；
3. $A = (\text{轨迹 reward} - \text{组内均值}) / \text{std}$ ，**整条轨迹的每一步、每个 token 用同一个 A**。

代价是信用分配很粗：分不清几十步里哪一步是关键。PPO + critic 能用 GAE 做步级分配，这里做不了。

关键区分：**多步 + 无 critic** 是这道题（提交 / 评测形式）的限制，不是 RL 方法本身的限制。换一个自己搭的多步 RL 场景，照样可以上 PPO + critic。

2.5 多步不等于组合爆炸

- **深度**：一条轨迹的步数由预算决定（100 次 tool call / 500 turns / 60 分钟），可以几十步，而不是两层。这是序列长度，不是树深。
- **不是树搜索**：GRPO 采的是 N 条**互相独立的线性轨迹**，每条从头走到尾，不在每一步分叉，所以没有组合爆炸。
- **不用 replay buffer**：GRPO / PPO 是 on-policy 方法，采一批 → 更新 → 丢弃 → 再采，不存经验池（那是 DQN 这类 off-policy 方法的做法）。

2.6 scaffold 和 RL 都要

scaffold 和 RL 不是二选一：scaffold（yaml + prompt + 工具编排）负责”接线”，RL / LoRA 负责训练决策权重。顺序是：

1. 最小 raw scaffold 跑 baseline 看分；
2. prompt tuning（像调超参，不动权重，迭代快）；
3. scaffold 不太差、有正样本之后再上 RL。

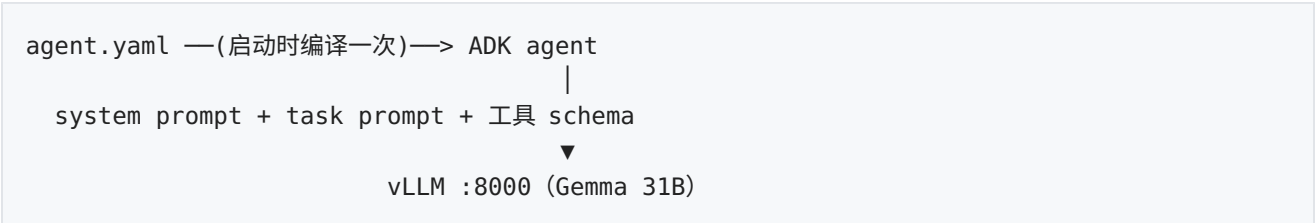
3. 系统全景：一个 task 是怎么跑完的

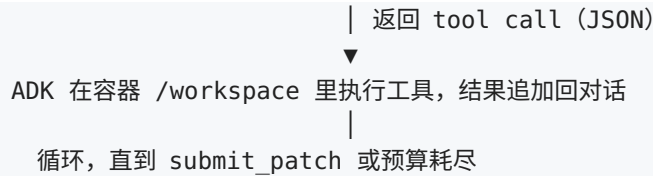
3.1 三个库 + 一个推理服务

组件	角色
swegemma	harness ：两容器生命周期、绑定 9 工具、生成 task prompt、抽 diff、pytest 判分。CLI 是 swegemma eval
adk-submission	把 agent.yaml 编译成 Google ADK agent（不执行任何选手代码），并能管理 vLLM 进程
adk-eval-core	底层：沙箱、三级容错 edit_file、token 计数、轨迹 trace
vLLM	把 Gemma 包成一个 OpenAI 兼容的 HTTP 服务（127.0.0.1:8000/v1）

一句话：**harness = 考场 + 监考 + 阅卷**。agent 是考生，模型是考生的脑子；我们能控制的只有考生（prompt、工具选择、采样参数、LoRA），harness 是官方固定的，本地和 Kaggle 跑的是同一套。

3.2 一次循环





几个容易混的点：

- **yaml 只在启动时用一次**，定义” agent 能做什么”。模型每一步输出的是 tool call (JSON)，名字必须在 yaml 的 tools: 里。
- **两条 prompt**: system prompt 是我们写的；第一条 user 消息 (task prompt) 是 harness 里的 build_agent_prompt 用固定模板 + 仓库名 + issue 原文 + 环境提示生成的，我们改不了。
- **/workspace 在容器里**：每个 task 起一个新容器，把 snapshot 解压到 /workspace，结束即销毁。
- agent 没调 submit_patch 时，harness 会自动抽 diff 兜底；submit_patch 和 get_status 不计入 tool call。
- scratch 脚本要放 /tmp，放在 /workspace 会被打进 patch。

3.3 数据：snapshot / graph / embedding

- **snapshot** (.tgz，每题一份，约 220MB)：该 task 在 base_commit 时的完整 git repo，用 git fast-export 截断，**没有未来提交**。
- **graph** (NetworkX JSON，按 commit 存)：主办方预先用静态分析生成的代码关系图。点是模块 / 类 / 函数，边是 CALLS / DEFINED_IN / IMPORTS。供 get_code_neighbors / get_code_subgraph 使用。
- **embedding** (256 维 .npz，按 commit 存)：图上每个节点的向量，供 search_similar_code 使用。
- snapshot 129 个，graph / embedding 各 127 个：有少数 task 共用同一个 commit。
- tasks.jsonl 每行：instance_id / repo / base_commit / patch (gold) / test_patch / problem_statement / hints_text / created_at。gold patch 和 test_patch 只能用于分析，不能进 prompt。

4. 环境搭建实录

4.1 机器选择

- **RunPod 不行**：pod 是容器，内核级 namespace / capability 受限，跑不了 docker。
- **Lambda Labs**：完整 VM，A100-SXM4-40GB，\$1.99/hr。docker + GPU 验证通过，可以本地跑完整 harness，不用等 Kaggle 后台。
- **持久化**：Persistent filesystem 挂在 /lambda/nfs/gemma-comp (\$0.20/GB/月)，terminate 实例后保留。本地盘和 /var/lib/docker 会丢。

原则：数据、wheelhouse、venv、模型权重全放 NFS；换机器只跑一遍 bootstrap.sh。

4.2 步骤与坑

步骤	坑 / 结论
列竞赛文件	slug 是 <code>gemma-4-developer-agent</code> ，猜错名字会 404；文件列表要翻页（524 个，6 页）
下小文件	<code>README</code> 、 <code>tasks.jsonl</code> 、 <code>docker/</code> 、 <code>sandbox/</code> 、 <code>sample_submission/</code> 共 17 个
读 <code>sample_submission</code>	是完整示范版：主 agent + <code>code_analyzer</code> 子 agent，两个 LoRA 是约 200KB 的占位假 adapter； <code>eval_config</code> 是 1 分钟 / 10 次 tool call 的冒烟配置； <code>system.md</code> 是约 4KB 的详细工作流——都不是 raw
装 harness	三个库 不在 PyPI ，在 Kaggle 数据集 <code>metric/gemma-4-developer-agent-wheelhouse</code> 里。要用 <code>--find-links</code> ，不能 <code>--no-index</code> （wheelhouse 不含公开依赖）
Python 版本	系统是 3.10，用 uv 建 3.12 venv
venv 放 NFS	uv 缓存和 venv 不在同一文件系统，无法硬链接，退化成全量拷贝，装 <code>swegemma</code> 用了 6 分钟。权衡后仍放 NFS：只慢一次，换机器省得多
vLLM	不是 <code>swegemma</code> 的依赖， <code>wheelhouse</code> 里单独带了 0.19.1。装上后 torch 从 2.14.1 降到 2.10.0+cu128；驱动 580，CUDA OK
模型权重	HF <code>google/gemma-4-31b-it-qat-w4a16-ct</code> ，23.3GB，未 gated。HF_HOME 设到 NFS，约 1 分钟下完
起 vLLM 报错	<code>transformers</code> 被 <code>swegemma</code> 带到 5.18.0，把 <code>head_dim</code> 改成按层属性，vllm 0.19.1 读 <code>config.head_dim</code> 直接报错。按发布日期选 vllm（04-18）之前最近的 5.5.4 （04-13）解决
本地谁起 vLLM	源码确认：本地 <code>swegemma eval</code> 不起 vLLM，默认连 <code>localhost:8000/v1</code> ，并把模型名发成 <code>openai/gemma-4-31b-it-qat-w4a16-ct</code> 。所以自己起， <code>--served-model-name</code> 必须对上
启动参数	直接调 <code>adk_submission.server.VllmServer.build_cmd()</code> 生成命令，和 Kaggle 一致（tool / reasoning parser = <code>gemma4</code> 、开 <code>thinking</code> 、32K），只改 TP=1，再补 <code>served name</code>
docker 权限	用户不在 docker 组；harness 自己要连 docker，所以不能靠 <code>sudo</code> 绕过
构建镜像	<code>Dockerfile.sandbox</code> 的 build context 是 <code>docker/</code> 目录。默认镜像名 <code>swebench-sandbox:latest</code>
离线 wheels	容器里给被测 repo 装依赖要用竞赛的 <code>wheels/</code> （124 个），要单独下载

4.3 显存账（A100-40G，单卡）

Kaggle 是 4×L4，每卡 24G。23G 权重单卡装下后几乎不剩 KV cache 的空间，所以用 `tensor parallel = 4`：把同一个模型的每层切成 4 份，4 张卡一起算一个 token。A100-40G 单卡放得下，TP=1，行为一致。

vLLM 实测日志：

项	值
权重显存	19.62 GiB
KV cache 可用	14.71 GiB (约 16,064 token)
32K 请求最大并发	3.03×
权重加载	175 s (NFS 读约 130MB/s)
profile + warmup	131 s
注意力后端	head_dim 不一致 (256 / 512) , 强制 TRITON_ATTN

结论：40G **够跑推理**（baseline、prompt tuning，本地并发 1-2），**不够 RL 训练**（底座 + 长轨迹激活 / 梯度，还要和 rollout 抢卡）。RL 阶段换 80G 或多卡，环境在 NFS 上，只需跑 bootstrap。

5. 概念速查（大白话）

- **vLLM**：把 LLM 当一个 service 用。加载权重到 GPU，对外提供 OpenAI 格式的 HTTP 接口；agent 每走一步就发一次请求。优势是 PagedAttention、连续批处理、multi-LoRA、prefix caching。
- **Uvicorn / FastAPI**：FastAPI 定义”哪个 URL 路径怎么响应”，并按类型注解校验请求；Uvicorn 是真正监听端口的服务器。日志出现 Uvicorn running / Application startup complete 就说明服务可以用了。（题库里的 fastapi task 修的就是这个库本身。）
- **CUDA graph**：把一次前向要发射的几百个 GPU kernel 提前录成一个”宏”，之后 CPU 只说一句”播放”。GPU 负责算，但每一步算什么都要 CPU 下指令；小 batch 时 CPU 发指令才是瓶颈。启动时那一两分钟就在录这些宏。
- **docker 镜像 / 容器**：镜像像装好系统的”系统盘快照”，容器是用它启动的运行实例。类似 Chrome 开一个 tab：互相隔离、共用同一个内核，所以秒级启动，比虚拟机轻。
- **harness**：运行框架 + 测试框架。

6. Raw baseline

```
name: raw_agent
model: gemma-4-31b-it-qat-w4a16-ct
instruction: !include prompts/system.md
tools: [run_command, read_file, edit_file, write_file, get_status,
        submit_patch, get_code_neighbors, search_similar_code,
        get_code_subgraph]
generate_content_config: !include configs/sampling.yaml
```

- system prompt 只有一句：你是软件工程师，修改 /workspace 下的源码解决 issue，做完调 submit_patch。只给身份、目标、结束动作，不给工作流，测模型的裸能力。
- sampling 沿用官方配置：temperature 0.2, top_p 0.95, max_output_tokens 16384, thinking_budget 4096。
- 不带 adapter、子 agent 和 eval_config；预算用命令行参数对齐 Kaggle：100 次 tool call、60 分钟。

- 冒烟验证：直接请求 vLLM，tool call 能正确解析成 `run_command({"command": "ls /tmp"})`。
- 第一个 task：fastapi_15661（tasks.jsonl 第一行，只为打通流程），已在跑，system / task prompt 注入成功。

结果待补：本稿写到这里时第一题还在跑。

7. 下一步

1. 拿到 fastapi_15661 的结果，再跑全量 129 题 raw baseline 的 Resolution Rate。
2. 写 debug_case.py：给定 instance_id，解压 repo，并排放 gold / test / agent 三份 patch；把 trace 转成” thinking → tool call → 结果” 的可读 markdown；用同一镜像起交互容器复现判分。
3. prompt tuning，按 case 分析迭代。
4. scaffold 有正样本后上 GRPO + LoRA，换 80G 卡。

附录：NFS 布局与 bootstrap

```
/lambda/nfs/gemma-comp/
├─ bootstrap.sh           # 新机器恢复环境
├─ secrets/kaggle.json
├─ venv/                  # py3.12: swegemma 0.2.7、vllm 0.19.1、transformers 5.5.4
├─ wheelhouse/           # 官方 harness wheel
├─ hf/                   # HF_HOME, 模型权重
├─ data/                 # tasks.jsonl、snapshots/、graphs/、embeddings/、wheels/、
docker/ ...
├─ submissions/raw_v0/
├─ results/raw_v0/       # summary.json、patches/、traces/、logs/
├─ repos/                # 给人看的解压 repo
└─ logs/vllm.log
```

bootstrap.sh 做的事：装 uv 和同版本 Python 3.12.15（venv 里的 python 是指向 ~/.local 的软链，换机器会断）→ 恢复 kaggle 凭证 → 写环境变量（HF_HOME、PATH 等）→ docker 权限 → 镜像不存在就 build → 确认 transformers 是 5.5.4 → 自检。前提是新机器在同一区域、挂上同一个 filesystem；跨区域就要全部重来。